

Efficient Long CoT Reasoning in Small Language Models

Zhaoyang Wang¹† Jinqi Jiang¹† Tian Qiu¹† Hui Liu²

Xianfeng Tang² Huaxiu Yao¹

¹University of North Carolina at Chapel Hill ²Amazon

{zhaoyang, huaxiu}@cs.unc.edu

Abstract

Recent large reasoning models such as DeepSeek-R1 exhibit strong complex problems solving abilities by generating long chain-of-thought (CoT) reasoning steps. It is challenging to directly train small language models (SLMs) to emerge long CoT. Thus, distillation becomes a practical method to enable SLMs for such reasoning ability. However, the long CoT often contains a lot of redundant contents (e.g., overthinking steps) which may make SLMs hard to learn considering their relatively poor capacity and generalization. To address this issue, we propose a simple-yet-effective method to prune unnecessary steps in long CoT, and then employ an on-policy method for the SLM itself to curate valid and useful long CoT training data. In this way, SLMs can effectively learn efficient long CoT reasoning and preserve competitive performance at the same time. Experimental results across a series of mathematical reasoning benchmarks demonstrate the effectiveness of the proposed method in distilling long CoT reasoning ability into SLMs which maintains the competitive performance but significantly reduces generating redundant reasoning steps.

1 Introduction

Chain-of-thought (CoT) prompting (Wei et al., 2022b; Wang et al., 2023a; Kojima et al., 2022) has emerged as an effective approach for enhancing the reasoning capabilities of large language models (LLMs). Explicitly prompting models with phrases such as “Let’s think step by step” and incorporating CoT-rich data during training have already become standard practices (Meta-AI, 2024; Abdin et al., 2024; Qwen-Team, 2024; Liu et al., 2024; Team et al., 2024). Recent advancements in complex reasoning as OpenAI’s o1 (OpenAI, 2024), Qwen-QwQ (Team, 2025b), and DeepSeek-R1 (Guo et al., 2025) large reasoning models further demonstrate that scaling up the length of CoT steps can significantly improve model performance in solving

† Equal contribution.

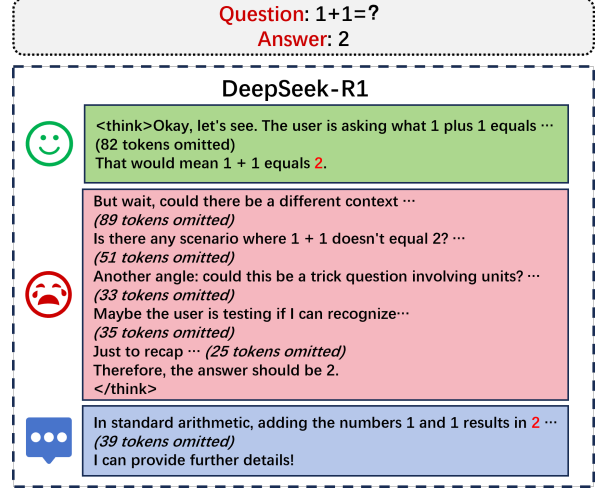


Figure 1: Illustration of redundant reasoning to a simple question by DeepSeek-R1 (Guo et al., 2025). The green part is sufficient for a correct answer, the red part is redundant, and the blue part is the summary response.

complex reasoning problems. While these efforts extend the boundaries of what LLMs can achieve, they also introduce new challenges to small language models (SLMs) with about 7B parameters which often use distillation methods to learn such long CoT reasoning (Guo et al., 2025; Face, 2025).

While long CoT is necessary in scaling the performance of reasoning, its increasing length introduces significant computational inefficiency. Some recent works (Chen et al., 2025; Aggarwal and Welleck, 2025; Yang et al., 2025; Zhang et al., 2025) have found that generated long CoT traces often contain many redundant reasoning steps even to the very simple question, as shown in Figure 1. Those redundant reasoning steps may not only bring unnecessary computation burden during test time, but also affect the reasoning performance (Sui et al., 2025; Aggarwal and Welleck, 2025; Wu et al., 2025; Marjanović et al., 2025). Moreover, long CoT with redundant reasoning steps can make barriers to the distillation process, since SLMs have relatively poor capability and generalization.

To address this issue, existing works propose to use heuristic rules such as minimum reasoning length with correct final answer (Chen et al., 2025), design length based rewards for reinforcement learning (Aggarwal and Welleck, 2025; Yi and Wang, 2025; Yang et al., 2025), or advanced prompting methods (Wu et al., 2025; Munkhbat et al., 2025; Xia et al., 2025; Han et al., 2025; Nayab et al., 2025). However, these methods either rely on re-designing the rewards during reinforcement learning which is often less effective than direct distillation from larger reasoning models (Guo et al., 2025; Face, 2025) and requires more computation resources, or do not consider the target SLM’s reasoning ability when selecting the long CoT training data. This leads to the question: *How can high-quality CoT traces generated by large reasoning models be efficiently distilled into SLMs?*

In this paper, we first observed that long CoT reasoning generated by large reasoning models such as DeepSeek-R1 (Guo et al., 2025) often contains unnecessary reasoning steps, even for the simple questions such as “What is 1+1?”. This observation suggests that SLMs may not need to learn the entire CoT reasoning process, but rather focus on the essential reasoning steps that lead to the correct answer. Motivated by this, we propose a simple yet effective method to prune redundant reasoning steps in generated long CoT data. Specifically, We use binary cutting to efficiently search for the shortest concise CoT steps that lead to the correct answer, which greatly reduces the searching space and time complexity. Further, we noticed that SLMs can often directly infer the correct answer from partial long CoT steps, and those segments of CoT vary with SLMs’ own capabilities. Based on this, we propose an on-policy distillation method, in order to enhance the aforementioned binary cutting method to search the most tailored segments of long CoT for the target SLM. Finally, we utilize these tailored concise CoT data to fine-tune the target SLM for complex reasoning with supervised fine-tuning (SFT) and direct preference optimization (DPO) (Rafailov et al., 2023). Experimental results across a series of mathematical reasoning tasks demonstrate the effectiveness of the proposed method in distilling long CoT reasoning ability into SLMs while significantly reducing the generation of redundant reasoning steps, which leads to a more efficient long CoT reasoning paradigm for SLMs.

In summary, our contributions are three-fold:

- 1) We observed that long CoT reasoning generated by large reasoning models often contains unnecessary reasoning steps, which is then identified to be harmful for distilling such complex reasoning ability into SLMs.
- 2) We propose a simple yet effective method to prune redundant reasoning steps in long CoT, which uses binary cutting for efficiently searching and on-policy validation that adapts to the target SLM’s capability.
- 3) Experiments and analysis demonstrate the effectiveness of our method in enabling SLMs with efficient long CoT reasoning, which significantly reduces generating redundant reasoning steps while preserving performance.

2 Related Work

Chain of Thought. Chain of thought (CoT) reasoning has been widely adopted to enable LLMs to perform reasoning in a step-by-step manner (Wei et al., 2022a,b; Kojima et al., 2022; Wang et al., 2023a; Zhou et al., 2023). Recently, the emergence of large reasoning models such as OpenAI’s o1 series (OpenAI, 2024), DeepSeek-R1 (Guo et al., 2025), Qwen’s QwQ (Team, 2025b) and Kimi’s k1.5 (Team, 2025a) has demonstrated that scaling up the length of CoT reasoning can further improve model performance on complex reasoning tasks, which calls for the need to empower SLMs with such long CoT reasoning ability.

Redundancy in Long CoT. Large reasoning models often exhibit an overthinking problem, generating unnecessary or repetitive CoT steps that inflate sequence length which can lead to inefficiency and even harm final answer accuracy (Sui et al., 2025; Wu et al., 2025; Marjanović et al., 2025; Chen et al., 2025). To mitigate this issue, existing works have explored several approaches such as heuristic pruning methods that truncate CoT to the minimal prefix yielding a correct answer (Chen et al., 2025), reinforcement-learning approaches that incorporate length-based penalties into the reward function (Aggarwal and Welleck, 2025; Yang et al., 2025; Yi and Wang, 2025), and alternative prompting techniques that guide models toward more concise reasoning (Wu et al., 2025; Xia et al., 2025; Han et al., 2025; Nayab et al., 2025). However, these methods often ignore the fact that different SLMs vary in their reasoning capabilities. In contrast, our approach combines a simple binary-cutting algorithm with an on-policy

search tailored to the target SLM. This enables an efficient $O(\log n)$ search for concise long CoT segments while adapting to the capability of the small model, ensuring both efficiency and performance in long CoT reasoning.

LLM Distillation. Distilling knowledge from LLMs into open-source SLMs has proven to be a simple yet effective approach to empower SLMs with new capabilities (Hinton et al., 2015; Xu et al., 2024). Beyond this, research community has successfully extended distillation to reasoning tasks by fine-tuning SLMs on rich CoT data annotated by LLMs (Ho et al., 2023; Fu et al., 2023; Magister et al., 2023; Wang et al., 2023b; Shridhar et al., 2023). More recently, OpenR1 (Face, 2025) and DeepSeek-R1’s SLMs series (Guo et al., 2025) have demonstrated that SLMs can be effectively trained to perform complex reasoning via long CoT distillation. However, these efforts overlook the detrimental effects of redundant or unnecessary reasoning steps within the CoT process, which can overwhelm SLMs’ limited capacity and generalization. In this paper, we focus on pruning such redundancy to enable efficient long CoT distillation and tailor the data to the target SLM.

3 Method

3.1 Background

Large reasoning models (e.g., DeepSeek-R1) are capable of generating long CoT reasoning steps to solve complex problems. However, these models usually produce excessive reasoning steps, where many of them are devoted to repeatedly verifying or reconfirming the correctness of an already correct answer. This behavior leads to unnecessarily long outputs and may also be harmful for the reasoning performance (Marjanović et al., 2025).

The goal of this paper is to enable open-source SLMs to take advantage of long CoT reasoning ability while maintaining efficiency. To achieve this, most of existing works introduce length based penalties into the reward function (Aggarwal and Welleck, 2025; Yang et al., 2025; Yi and Wang, 2025), then train SLMs with similar reinforcement learning adopted by DeepSeek-R1 (Guo et al., 2025). However, this approach is often less effective than direct distillation from larger reasoning models (Guo et al., 2025) in terms of both performance and training efficiency. Thus, we choose to search segments of the long CoT reasoning that

meet two constraints: (i) *valid*, it should be sufficient for the target SLM to generate the correct final answer, and (ii) *efficient*, it should be as short as possible to maximize efficiency. Then, we can use those long CoT segments to fine-tune the target SLM to achieve efficient long CoT reasoning.

In the following sections, we will first present the curation of streamlined long CoT data which includes the proposed binary cutting method and on-policy validation method, and then detail the fine-tuning process for the target SLM. The whole framework is illustrated in Figure 2.

3.2 Streamlining Long CoT

3.2.1 Response Sampling

To collect long CoT data, we first sample a set of long CoT reasoning responses generated by the large reasoning models. Given the input question Q , the complete response R includes two parts as shown in Figure 1: (1) the thinking part T , which is the intermediate reasoning process enclosed within special `<think>` and `</think>` tags, and (2) the final response part, which is the summary paragraph to the thinking part and contains the final answer to the question. The thinking part is often natural paragraph structure, and each paragraph can be viewed as a reasoning step. We split the thinking part T into a list of reasoning steps $T = [s_1, s_2, \dots, s_n]$, and we will operate on the step level for the subsequent methods. Combined with the ground truth final answer A , we can obtain a dataset of triplets $D_{\text{original}} = \{(Q_i, T_i, A_i)\}_{i=1}^N$. The objective is to find a subset of reasoning steps $T_i^{j:k}$ for each example i such that maximizes the efficiency of the reasoning process while ensuring the correctness of the final answer, where $[j : k]$ means the set of steps $[s_j, s_{j+1}, \dots, s_k]$.

3.2.2 Binary Cutting

Previous work (Chen et al., 2025) proposes First-Correct Solutions (FCS), which truncates the reasoning process to the minimal prefix yielding a correct answer. FCS requires checking every prefix $T^{1:k} = [s_1, \dots, s_k]$ for $k = 1, \dots, n$. While this linear search requires $O(n)$ complexity, an unconstrained search over all 2^n subsequences would be intractable and break the reasoning chain’s coherence. To maintain contiguity yet improve efficiency, we likewise restrict our search to prefixes, but replace the linear search with a binary cutting strategy. This reduces the computation complexity to $O(\log_2 n)$ while still guaranteeing the discovery

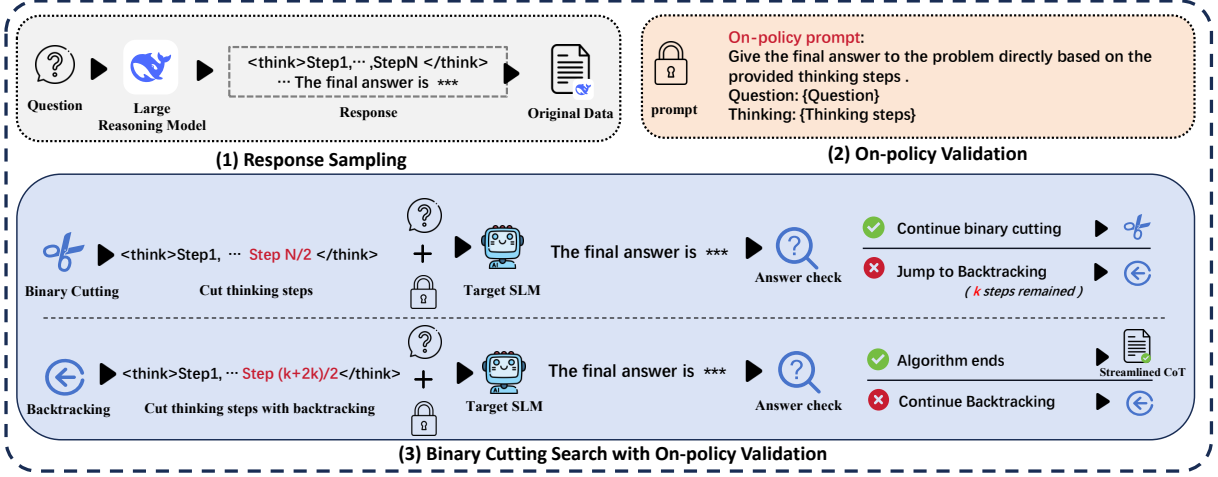


Figure 2: Overview of the proposed streamlining long CoT method which includes 3 key stages for data curation. (1) Response sampling that samples original long CoT reasoning samples from the large reasoning model. (2) On-policy Validation which prompts the target SLM to generate the final answer based on the segments of the reasoning (thinking). (3) Binary Cutting Search with On-policy Validation that combines binary cutting and on-policy validation to search valid streamlined long CoT reasoning steps, in order to fine-tune the target SLM.

of the minimal valid prefix. Specifically, at each iteration we compute $m = \lfloor (low + high)/2 \rfloor$, truncate the original reasoning chain $T = [s_1, \dots, s_n]$ to the prefix $T^{1:m}$, and invoke the validation function $\phi(Q, T^{1:m}, A)$ which decides whether the segment is good and valid. If the prefix still yields the correct answer ($\phi = 1$), we update $best \leftarrow T^{1:m}$ and set $high \leftarrow m$ to search even shorter prefixes. However, if the prefix fails to yield the correct answer ($\phi = 0$), we introduce a backtracking mechanism to recover the last valid prefix: resetting low to the last mid-point and perform a binary search upward toward n , computing $m = \lceil (low + n)/2 \rceil$ at each step until ϕ returns true again. The proposed binary cutting method with backtracking mechanism first aggressively cutting, which guarantees that the selected steps form a contiguous prefix, and recovers any essential steps that might have been over-pruned. In contrast, as 61.83% of the samples contain the answer in the last 10 steps, cutting from the beginning is not effective for SLM training. This approach effectively reduces the search space and time complexity to $O(\log_2 n)$, while ensuring that the selected reasoning steps are both valid and efficient.

3.2.3 On-Policy Validation

Existing pruning methods such as FCS assume a single “oracle” validation criterion, typically provided by an additional judge model, to judge whether a truncated CoT remains correct. However, this ignores the fact that different SLMs ex-

hibit distinct reasoning biases and strengths. To generate training data that is tailored to the target SLM’s own inductive preferences, we let the SLM M itself serve as the validator in an on-policy paradigm. Specifically, we construct a specialized prompt P_{policy} (see Figure 2, part (2)) which asks model M to produce the final answer given only the question Q and a candidate prefix $T^{1:k}$. We then can define the validation function ϕ :

$$\phi(Q, T^{1:k}, A; M) = \mathbf{1}\{M_t(Q, T^{1:k}, P_{policy}) = A\}. \quad (1)$$

During binary cutting, each prefix is accepted only if $\phi = 1$ under M . By relying on the target model’s own outputs rather than an external judge model, we ensure that the distilled CoT segments align with the SLM’s native reasoning capacity. This on-policy mechanism overlooked by prior methods yields a more coherent long CoT segment, since each retained prefix is one that the SLM can already interpret correctly. The whole method for streamlining long CoT is illustrated in Algorithm 1. See Appendix C for the full on-policy prompt.

3.3 Fine-tuning SLM

After applying binary cutting and on-policy validation methods, we obtain a distilled dataset $D_{distill} = \{(Q_i, R_i, Y_i)\}_{i=1}^N$, where R is the original whole response, and Y is the pruned concise response containing the pruned thinking part and the final response part. We can use this dataset $D_{distill}$ to fine-tune SLM to learn efficient long CoT reasoning via

Algorithm 1 Streamlining Long CoT.

Require: Triplet (Q, T, A) where $T = [s_1, \dots, s_n]$, target model M , Validate function ϕ

Ensure: Shortest valid contiguous long CoT segment $T^{1:k}$

```
1:  $low \leftarrow 1, \quad high \leftarrow n, \quad best \leftarrow T^{1:n}$ 
2: while  $low < high$  do
3:    $mid \leftarrow \lfloor (low + high)/2 \rfloor$ 
4:    $T' \leftarrow T^{1:mid}$ 
5:   if  $\phi(Q, T', A, M)$  is true then
6:      $best \leftarrow T'$ 
7:      $high \leftarrow mid$   $\triangleright$  can still shorten
8:   else
9:     break  $\triangleright$  enter backtracking
10:  end if
11: end while
12:  $low \leftarrow mid, \quad high \leftarrow n$   $\triangleright$  Backtracking:
    recover any over-pruned steps
13: while  $low < high$  do
14:    $mid \leftarrow \lceil (low + high)/2 \rceil$ 
15:    $T' \leftarrow T^{1:mid}$ 
16:   if  $\phi(Q, T', A, M)$  is true then
17:     return  $T'$   $\triangleright$  found minimal valid
        prefix
18:   else
19:      $low \leftarrow mid + 1$ 
20:   end if
21: end while
22: return  $best$ 
```

supervised fine-tuning (SFT) and direct preference optimization (DPO) (Rafailov et al., 2023).

3.3.1 SFT Training

The most straightforward approach to leverage the obtained distilled data is to apply SFT training on the target model. Given the distilled dataset D_{distill} , the target SLM M is fine-tuned to maximize the likelihood of the pruned reasoning R conditioned on the input question Q as follows:

$$\mathcal{L}_{\text{SFT}} = -\mathbb{E}_{(Q,Y) \sim D_{\text{distill}}} \log M(Y|Q). \quad (2)$$

After training, the SLM is expected to generate more concise and efficient reasoning steps while maintaining the correctness of the final answer.

3.3.2 DPO Training

To help the target model better distinguish between “good” and “bad” reasoning steps, we can leverage

preference learning methods such as DPO to further fine-tune the model. Here, “good” refers to the pruned response with concise reasoning steps, while “bad” refers to the original response with redundant reasoning steps. The DPO training objective can be formulated as follows:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(Q,R,Y) \sim \mathcal{D}_{\text{distill}}} \left[\log \sigma \left(\beta \log \frac{M(Y|Q)}{M_{\text{ref}}(Y|Q)} - \beta \log \frac{M(R|Q)}{M_{\text{ref}}(R|Q)} \right) \right], \quad (3)$$

where $\sigma(\cdot)$ denotes the logistic function, $\beta = 0.1$ is a hyperparameter of DPO, and M_{ref} is the frozen reference model typically the SLM after SFT training. Thanks to the significant difference between two responses R and Y in terms of the response length, the DPO training can effectively help the target SLM to learn the preference of concise reasoning steps over redundant ones.

4 Experiments

4.1 Experimental Setup

Datasets. To evaluate not only the reasoning performance but also the efficiency of long CoT reasoning, we benchmark SLMs on three mathematical reasoning datasets of increasing difficulty: GSM8K (Cobbe et al., 2021), MATH (Hendrycks et al., 2021), and AIME (MAA, 2024). GSM8k is a primary school level mathematical dataset requiring basic arithmetic and logic. MATH we used is a widely used subset of original dataset which contains 500 challenging high school competition-level math problems. AIME consists of extremely difficult math problems spanning from 1983 to 2025, aiming to test model’s generalization ability.

Models. We conducted experiments mainly on two popular open-source SLMs, Llama-3.1-8B-Instruct (Meta-AI, 2024) and Qwen2.5-7B-Instruct (Qwen-Team, 2024), which do not originally own the long CoT reasoning ability.

Baselines Methods. In experiments, we mainly compare our method with the following baseline methods: (1) “Base”, using the original SLM, as the baseline performance. (2) “Full”, as described in Guo et al. (2025), we directly use the original long CoT which may contain redundant reasoning steps for training, in order to directly show the necessity to remove such redundant steps. (3). “Short CoT”, using the normal CoT data without scaling length to train the SLM, in order to demonstrate the effectiveness of scaling CoT. (4). “FCS”,

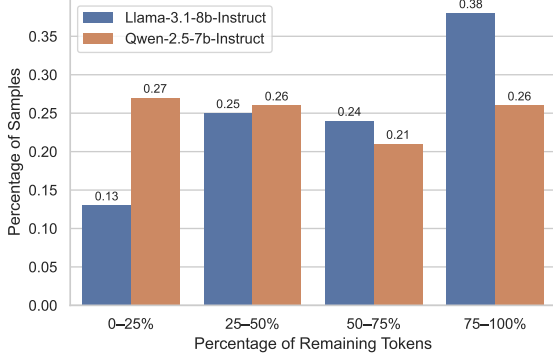


Figure 3: Distribution of the remaining tokens ratio across different percentage intervals after streamlining.

First-Correct Solutions strategy (Chen et al., 2025) which linearly search the first occurred segment of long CoT with the correct final answer, in order to demonstrate the superiority of our method in pruning unnecessary reasoning steps and enhancing long CoT of SLMs. Following original implementations, we use a LLM Qwen2.5-14B-Instruct (Qwen-Team, 2024) to segment the thinking part instead of nature line break.

Implementation Details. We use an existing large scale prepared long CoT data D_{original} OpenR1-Math-220k (Face, 2025)¹, which has 220k math problems from NuminaMath 1.5 (LI et al., 2024), each paired with two to four long CoT reasoning generated by DeepSeek-R1 (Guo et al., 2025). To form the distilled dataset, we perform binary cutting search with on-policy validation on the “train” split of this dataset which has about 93.7k samples. Finally, we have valid pruned long CoT data D_{distill} about 25k samples for training. In most cases, the target SLM is fine-tuned by SFT training in 3 epochs, and DPO training in 1 epoch, with a learning rate of $1e-6$. We also noticed that single DPO training can also decrease the likelihood of the “good” response, thus we add the SFT loss with a weight of 0.3 into Eq. 3 for stable performance. We use some regex expressions and format rules to extract the final answer from the generated response and calculate exact match accuracy.

4.2 Main Results

Table 1 shows the reasoning performance and efficiency of different methods.

Scaling CoT improves reasoning. All the long CoT based methods including “Full”, “FCS” and

“Ours” surpasses the model with “short CoT”, which validates the effectiveness of scaling CoT length in mathematical reasoning.

Long CoT contains redundant steps. First, we shot that, for the easy reasoning task GSM8K, “Full” requires an average of 1051 tokens for reasoning, while does not show obvious better performance than other methods which often need less than half of the generation. Figure 3 clearly shows that about 40% long CoT data can be streamlined over 50% redundant reasoning steps by our method. This supports the perspective that long CoT contains redundant steps especially for the easy tasks. Then, compared to “Full”, our method significantly decreases the number of generated thinking tokens, while only slightly decreases the reasoning performance for most cases. For example, our Llama achieves 54.00% accuracy on MATH after SFT, slightly lower than full long-CoT SFT (54.80%), while reducing average number of tokens by 29.07%. Qwen model after distillation exhibits a modest decrease in accuracy compared to “Full” on AIME dataset, that might because Qwen is strong enough on this dataset as the base model.

Our method achieves efficient long CoT. Compared to the baseline method “Full” and another long CoT pruning method “FCS”, ours often show better reasoning performance than “FCS”, while significantly increases the efficiency in long CoT reasoning. We also find that DPO training contributes the most to reducing the tokens, however, it harms the reasoning performance for hard tasks. We further stat the results when at the same on-policy condition in Table 2. For two target SLMs, Llama and Qwen, our method reduces the number of tokens to an average of 61.83% and 50.75%, respectively. However, the FCS and Random baseline methods retain more tokens, suggesting that the proposed binary cutting method is both efficient and effective in searching valid segments of long CoT reasoning. Additionally, We further evaluate the impacts of SFT loss weights in DPO training in Appendix B.

5 Analysis

5.1 Ablation Study

We conduct two types of ablation studies to evaluate the effectiveness of design choices of our method, shown in Table 3. First, we want to show that our binary cutting with backtracking mecha-

¹<https://huggingface.co/datasets/open-r1/OpenR1-Math-220k>

Table 1: Main Results. Note that only Base and Short CoT report the full token counts, while all other methods count only tokens within <think>...</think>. The subscripts of Acc and Token indicate the percentage of decline or rise of the model relative to the model with SFT_{Full} method.

Model	Method	GSM8K		MATH		AIME	
		Acc(%)	#Token	Acc(%)	#Token	Acc(%)	#Token
Llama	Base	76.80	232	37.80	2454	12.97	6204
	SFT _{Full}	89.01	1051	54.80	3274	16.51	7262
	SFT _{Short CoT}	60.05 _(32.54%↓)	314 _(70.12%↓)	23.40 _(57.30%↓)	1522 _(53.51%↓)	4.82 _(70.81%↓)	2269 _(68.76%↓)
	SFT _{FCS}	85.52 _(3.92%↓)	728 _(30.73%↓)	47.20 _(13.87%↓)	1769 _(45.97%↓)	10.08 _(38.95%↓)	2874 _(60.42%↓)
	SFT+DPO _{FCS}	87.57 _(1.62%↓)	598 _(43.10%↓)	50.20 _(8.39%↓)	1392 _(57.48%↓)	13.40 _(18.84%↓)	2135 _(70.60%↓)
	SFT _{Ours}	87.34 _(1.88%↓)	502 _(52.24%↓)	54.00 _(1.46%↓)	2322 _(29.08%↓)	18.01 _(9.09%↑)	5480 _(24.54%↓)
	SFT+DPO _{Ours}	87.41 _(1.80%↓)	339 _(67.75%↓)	52.40 _(4.38%↓)	1324 _(59.56%↓)	17.90 _(8.42%↑)	3779 _(47.96%↓)
Qwen	Base	83.70	280	59.40	600	21.76	1121
	SFT _{Full}	90.37	1011	64.00	2712	28.51	6330
	SFT _{Short CoT}	64.67 _(28.44%↓)	125 _(87.64%↓)	43.20 _(32.50%↓)	487 _(82.04%↓)	13.61 _(52.26%↓)	987 _(84.41%↓)
	SFT _{FCS}	56.79 _(37.16%↓)	384 _(62.02%↓)	30.40 _(52.50%↓)	1032 _(61.95%↓)	18.44 _(35.32%↓)	2615 _(58.69%↓)
	SFT+DPO _{FCS}	78.24 _(13.42%↓)	524 _(48.17%↓)	49.40 _(22.81%↓)	1914 _(29.42%↓)	21.22 _(25.57%↓)	4131 _(34.74%↓)
	SFT _{Ours}	89.16 _(1.34%↓)	382 _(62.22%↓)	61.20 _(4.38%↓)	901 _(66.78%↓)	25.51 _(10.52%↓)	3021 _(52.27%↓)
	SFT+DPO _{Ours}	89.92 _(0.50%↓)	278 _(72.50%↓)	56.60 _(11.56%↓)	489 _(81.97%↓)	21.54 _(24.45%↓)	1836 _(71.00%↓)

Table 2: Average token usage and remaining ratio by different streamlining methods. Note that the discrepancy in original token counts arises because, to ensure comparable training data across methods, only data deemed valid by the on-policy method is reused by FCS.

Method	Model	#Orig	#Remaining	Ratio
FCS	Llama	3659.45	2531.68	69.18%
	Qwen	3875.14	2695.19	69.55%
Random	Llama	4665.98	2333.51	50.01%
	Qwen	4919.19	2466.74	50.15%
Ours	Llama	3660.35	2263.17	61.83%
	Qwen	3875.96	1967.19	50.75%

nism can effectively identify redundant reasoning steps instead of randomly deleting. Thus, we introduce “Random” variant which randomly delete intermediate steps before applying either SFT or DPO training. From the results, we can find that random deletion leads to a substantial drop in accuracy across all datasets, indicating that preserving informative reasoning steps is essential for reasoning. Second, to assess the role of our on-policy validation, we train Llama using concise CoT generated by the Qwen model. The performance is clearly inferior to our on-policy method, highlighting the importance of policy alignment between SLM’s own reasoning capacity and long CoT data.

5.2 Quality of Streamlined CoT

We use GPT-4.1 as a LLM-as-a-Judge to automatically evaluate the quality of CoT reasoning stream-

lined by different methods. A reward-based scoring prompt available at Appendix C guides GPT-4.1 to rate each reasoning sample based on its correctness, completeness, conciseness and reasoning quality. We report the average scores from GPT-4.1 over 100 randomly selected examples. At the same time, we conduct human evaluation where annotators are asked to rank the four long CoT outputs into top-1 choice, middle two choice, and bottom one choice, based on overall reasoning quality and reasoning conciseness. This ranking scheme provides a coarse but interpretable assessment of human preference across different methods.

As shown in Table 4, we find that “Full” long CoT achieves the highest score due to its completeness. Among the other three streamlining methods, “Random” strategy performs poorly due to its natural drawback in logical coherence, while FCS receives lower scores in conciseness. Our method, while less complete than “Full”, achieves better balance between conciseness and logical consistency in reasoning. Human evaluations are generally consistent with the LLM-based evaluation: “Full” method is most frequently ranked at the top one due to its completeness, while our method receives significantly more top and middle rankings than the other baseline methods, thanks to its backtracking mechanism and on-policy validation.

5.3 Case Study

Figure 4 illustrates an example of streamlining a long CoT sample by our method with binary cutting

Table 3: Ablation study results. “Qwen data” means that Llama is trained using streamlined long CoT data tailored for Qwen SLM. The subscripts of Acc and Token indicate the percentage of decline or rise of the model relative to SFT_{Ours} and SFT+DPO_{Ours} respectively.

Model	Method	GSM8K		MATH		AIME	
		Acc (%)	#Token	Acc (%)	#Token	Acc (%)	#Token
Llama	SFT _{Ours}	87.34	502	54.00	2322	18.01	5480
	SFT _{Random}	85.44 _(2.18%↓)	995 _(98.21%↑)	47.20 _(12.59%↓)	2894 _(24.63%↑)	8.68 _(51.81%↓)	5771 _(5.31%↑)
	SFT _{Qwen data}	87.19 _(0.17%↓)	411 _(18.13%↓)	52.8 _(2.22%↓)	1896 _(18.35%↓)	16.08 _(10.72%↓)	5349 _(2.39%↓)
	SFT+DPO _{Ours}	87.41	339	52.40	1324	17.90	3779
	SFT+DPO _{Random}	76.42 _(12.57%↓)	121 _(64.30%↓)	34.60 _(33.97%↓)	193 _(85.42%↓)	5.79 _(67.65%↓)	323 _(90.44%↓)
	SFT+DPO _{Qwen data}	86.28 _(1.29%↓)	272 _(19.76%↓)	47.6 _(9.16%↓)	953 _(28.02%↓)	14.9 _(16.76%↓)	3006 _(20.46%↓)

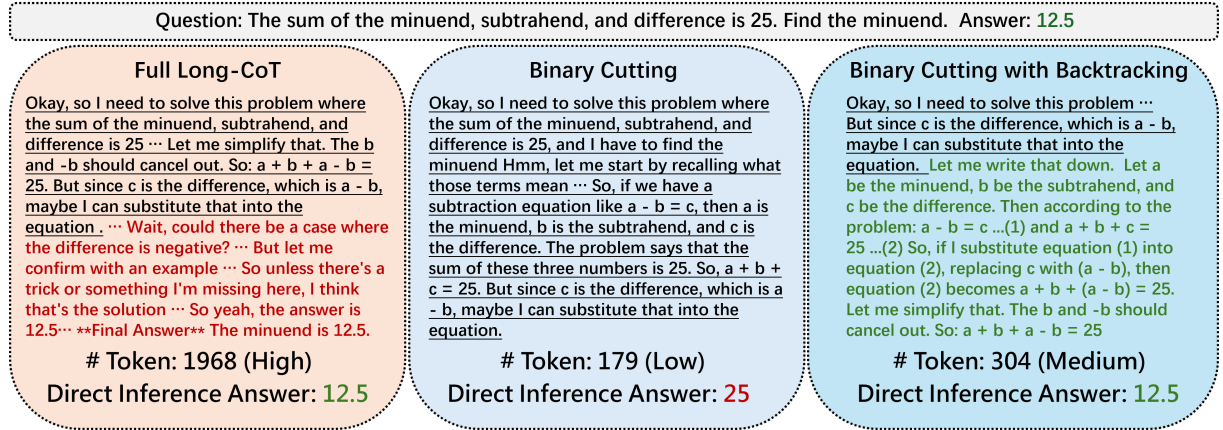


Figure 4: A case study of streamlining process of long CoT using our method. Underlined parts are the segments retained after binary cutting and are consistent across all three versions. Red text indicates steps removed during binary cutting. Green text marks reasoning steps restored in backtracking.

Table 4: LLM judgment and human preference ranking on different long CoT data. “T”, “M”, and “B” represent top one, middle two, and bottom one, respectively.

Method	LLM	Human		
		T%	M%	B%
Full	4.89	61	34	5
FCS	4.20	13	57	30
Random	4.24	2	54	44
Ours	4.44	24	55	21

and backtracking mechanism. The original full long CoT can prompt the SLM to infer the correct answer directly, but it comes with the overthinking issue. After several rounds of binary cutting, the over-concise CoT is no longer sufficient to make SLM arrive at the correct final answer. We are surprising to find that in the backtracking stage, our method restores part of previously removed steps, which successfully guides the SLM to generate the correct answer. More examples are in Appendix D.

6 Conclusion

In this paper, we tackle the challenge of distilling long CoT reasoning from large reasoning models into SLMs. We first identify that generated long CoT often introduces redundant or overthinking steps that waste computation during test time. To address this issue, we propose a binary cutting algorithm with backtracking, which locates the shortest contiguous prefix of the original reasoning that still yields a correct final answer in only $O(\log_2 n)$ complexity. Crucially, we further introduce on-policy validation that uses the target SLM itself as the judge of whether a truncated segment of reasoning remains valid and useful for generating the final answer, thereby adapting the distilled data to the SLM’s own reasoning strengths. Extensive experiments on multiple math reasoning datasets demonstrate that our approach preserves competitive reasoning performance while significantly reducing the redundant tokens in long CoT reasoning. We believe these findings make meaningful contributions to efficient long CoT reasoning in SLMs.

Limitations

In this section, we discuss the limitations of our paper while offering potentially useful advice for future research.

- 1) The proposed method uses a binary-cutting algorithm for efficient pruning of long CoT traces. While this search strategy is not guaranteed to find the globally optimal subset of reasoning steps, we believe it offers significant practical advantages in terms of efficiency. Exploring more optimal yet efficient search algorithms remains a direction for future work.
- 2) Our work focuses on the distillation scenario, which relies on a large reasoning model to provide high-quality reasoning traces. We do not consider reinforcement learning or self-training strategies for the SLM. While distillation offers high efficiency, alternative training paradigms are still valuable and complementary to our approach.
- 3) Due to limited computational resources, we evaluate our method only on two 7B-level models. We do not test across a wider range of model sizes. However, we believe 7B-scale models are the most commonly adopted among the community. Our framework should be applicable to any target model size.

References

- Marah Abidin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, et al. 2024. Phi-3 technical report: A highly capable language model locally on your phone. [arXiv preprint arXiv:2404.14219](#).
- Pranjal Aggarwal and Sean Welleck. 2025. L1: Controlling how long a reasoning model thinks with reinforcement learning. [arXiv preprint arXiv:2503.04697](#).
- Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, Dian Yu, Linfeng Song, Qiuzhi Liu, Mengfei Zhou, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. 2025. Do not think that much for $2+3=?$ on the overthinking of o1-like llms. [Preprint](#), arXiv:2412.21187.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. [Preprint](#), arXiv:2110.14168.
- Hugging Face. 2025. [Open r1: A fully open reproduction of deepseek-r1](#).
- Yao Fu, Hao Peng, Litu Ou, Ashish Sabharwal, and Tushar Khot. 2023. [Specializing smaller language models towards multi-step reasoning](#). In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 10421–10430. PMLR.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, and Ruoyu Zhang. 2025. [Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning](#). [Preprint](#), arXiv:2501.12948.
- Tingxu Han, Zhenting Wang, Chunrong Fang, Shiyu Zhao, Shiqing Ma, and Zhenyu Chen. 2025. [Token-budget-aware llm reasoning](#). [Preprint](#), arXiv:2412.18547.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. [Measuring mathematical problem solving with the math dataset](#). [Preprint](#), arXiv:2103.03874.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. [Distilling the knowledge in a neural network](#). [Preprint](#), arXiv:1503.02531.
- Namgyu Ho, Laura Schmid, and Se-Young Yun. 2023. [Large language models are reasoning teachers](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14852–14882, Toronto, Canada. Association for Computational Linguistics.
- Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. [Large language models are zero-shot reasoners](#). In *Advances in Neural Information Processing Systems*.
- Jia LI, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif Rasul, Longhui Yu, Albert Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. 2024. NuminaMath. [<https://huggingface.co/AI-MO/NuminaMath-1.5>](https://github.com/project-numina/aimo-progress-prize/blob/main/report/numina_dataset.pdf).
- Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. [arXiv preprint arXiv:2412.19437](#).
- MAA. 2024. [American invitational mathematics examination — aime](#). American Invitational Mathematics Examination – AIME 2024, February 2024.
- Lucie Charlotte Magister, Jonathan Mallinson, Jakub Adamek, Eric Malmi, and Aliaksei Severyn. 2023. [Teaching small language models to reason](#). In *Proceedings of the 61st Annual Meeting of the*

- Association for Computational Linguistics (Volume 2: Short Papers), pages 1773–1781, Toronto, Canada. Association for Computational Linguistics.
- Sara Vera Marjanović, Arkil Patel, Vaibhav Adlakha, Milad Aghajohari, Parishad BehnamGhader, Mehar Bhatia, Aditi Khandelwal, Austin Kraft, Benno Kroyer, Xing Han Lù, et al. 2025. Deepseek-r1 thoughtology: Let’s< think> about llm reasoning. [arXiv preprint arXiv:2504.07128](#).
- Meta-AI. 2024. Introducing meta llama 3: The next generation of open models. <https://ai.meta.com/blog/meta-llama-3-1/>. Accessed: 2025-05-18.
- Tergel Munkhbat, Namgyu Ho, Seo Hyun Kim, Yongjin Yang, Yujin Kim, and Se-Young Yun. 2025. Self-training elicits concise reasoning in large language models. [arXiv preprint arXiv:2502.20122](#).
- Sania Nayab, Giulio Rossolini, Marco Simoni, Andrea Saracino, Giorgio Buttazzo, Nicolamaria Manes, and Fabrizio Giacomelli. 2025. [Concise thoughts: Impact of output length on llm reasoning and cost](#). [Preprint](#), arXiv:2407.19825.
- OpenAI. 2024. [Learning to reason with llms](#). Accessed: 2025-03-05.
- Qwen-Team. 2024. [Qwen2.5: A party of foundation models](#).
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. [Advances in Neural Information Processing Systems](#), 36:53728–53741.
- Kumar Shridhar, Alessandro Stolfo, and Mrinmaya Sachan. 2023. [Distilling reasoning capabilities into smaller language models](#). In [Findings of the Association for Computational Linguistics: ACL 2023](#), pages 7059–7073, Toronto, Canada. Association for Computational Linguistics.
- Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Hanjie Chen, et al. 2025. Stop overthinking: A survey on efficient reasoning for large language models. [arXiv preprint arXiv:2503.16419](#).
- Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. 2024. Gemma 2: Improving open language models at a practical size. [arXiv preprint arXiv:2408.00118](#).
- Kimi Team. 2025a. [Kimi k1.5: Scaling reinforcement learning with llms](#). [Preprint](#), arXiv:2501.12599.
- Qwen Team. 2025b. [Qwq-32b: Embracing the power of reinforcement learning](#).
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023a. [Self-consistency improves chain of thought reasoning in language models](#). In [International Conference on Learning Representations](#).
- Zhaoyang Wang, Shaohan Huang, Yuxuan Liu, Jiahai Wang, Minghui Song, Zihan Zhang, Haizhen Huang, Furu Wei, Weiwei Deng, Feng Sun, and Qi Zhang. 2023b. [Democratizing reasoning ability: Tailored learning from large language model](#). [Preprint](#), arXiv:2310.13332.
- Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. 2022a. [Emergent abilities of large language models](#). [Transactions on Machine Learning Research](#). Survey Certification.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022b. Chain-of-thought prompting elicits reasoning in large language models. In [Advances in Neural Information Processing Systems](#).
- Tong Wu, Chong Xiang, Jiachen T Wang, and Praatek Mittal. 2025. Effectively controlling reasoning models through thinking intervention. [arXiv preprint arXiv:2503.24370](#).
- Heming Xia, Yongqi Li, Chak Tou Leong, Wenjie Wang, and Wenjie Li. 2025. [Tokenskip: Controllable chain-of-thought compression in llms](#). [Preprint](#), arXiv:2502.12067.
- Xiaohan Xu, Ming Li, Chongyang Tao, Tao Shen, Reynold Cheng, Jinyang Li, Can Xu, Dacheng Tao, and Tianyi Zhou. 2024. [A survey on knowledge distillation of large language models](#). [Preprint](#), arXiv:2402.13116.
- Junjie Yang, Ke Lin, and Xing Yu. 2025. Think when you need: Self-adaptive chain-of-thought learning. [arXiv preprint arXiv:2504.03234](#).
- Jingyang Yi and Jiazheng Wang. 2025. Shorterbetter: Guiding Reasoning Models to Find Optimal Inference Length for Efficient Reasoning. [arXiv](#).
- Jintian Zhang, Yuqi Zhu, Mengshu Sun, Yujie Luo, Shuofei Qiao, Lun Du, Da Zheng, Huajun Chen, and Ningyu Zhang. 2025. [Lightthinker: Thinking step-by-step compression](#). [Preprint](#), arXiv:2502.15589.
- Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V Le, and Ed H. Chi. 2023. [Least-to-most prompting enables complex reasoning in large language models](#). In [International Conference on Learning Representations](#).

A Release Plan

We will make our codebase, prompts, models and datasets publicly available after the camera-ready deadline to facilitate reproducibility and further research.

B DPO Training Settings

Table 5 presents the hyperparameters used in DPO training under different settings, including loss weights and data configurations.

Table 5: Experimental results of DPO with different SFT loss weights and data sizes.

Model+Method	SFT_weight / #Data	GSM8K		MATH		AIME	
		Acc (%)	#Token	Acc (%)	#Token	Acc (%)	#Token
Llama+DPO	0.1 / 5K	87.34	299	48.40	1102	15.65	2905
	0.2 / 5K	85.22	345	46.20	1397	14.79	3444
	0.3 / 5K	87.41	339	52.40	1324	17.90	3779
	0.1 / 10K	88.02	387	50.00	1399	17.04	3972
	0.1 / 20K	86.43	396	51.00	1651	17.90	4708
Qwen+DPO	0.1 / 5K	88.32	171	54.80	247	18.22	627
	0.2 / 5K	88.78	220	58.00	401	18.11	1078
	0.3 / 5K	89.23	251	55.60	443	19.94	1353
	0.1 / 10K	88.17	250	58.60	457	19.08	1409
	0.1 / 20K	89.92	278	56.60	489	21.54	1836

C Prompt Examples

We provide representative prompt examples used in different stages of our pipeline, including on-policy generation and evaluation with LLM-as-a-Judge.

On-policy Prompt. Figure 5 shows a typical prompt used to generate a final answer directly.

On-policy answering prompt

Given a mathematical problem and a series of thinking steps, your task is to determine the final answer based on the provided steps.

Instructions:

- Follow the given thinking steps exactly to derive the answer.
- Do not add any explanations, reasoning, or modifications.
- Output only the final answer in the specified format.

Formatting Requirement:

- The output should be strictly formatted as:

###Answer: <final_answer>

Now, here is the problem: {question}

The thinking steps are given below: {thinking_steps}

Figure 5: On-policy answering prompt format.

LLM-as-a-Judge Prompt. Figure 6 shows how we guide the LLM to score CoT based on predefined rules.

LLM-as-a-judge prompt

Review the user's question and the corresponding chain-of-thought (CoT) reasoning using the additive 5-point scoring system described below. Points are accumulated based on the satisfaction of each criterion:

- Add 1 point if the CoT is relevant and initiates a reasoning process related to the user's inquiry, even if it is incomplete or contains some irrelevant steps.
- Add another point if the CoT addresses a substantial portion of the reasoning needed for the question, but does not completely resolve it or contains gaps in logic.
- Award a third point if the CoT outlines the basic logical structure needed to solve the problem, demonstrating generally sound reasoning without unnecessary tangents.
- Grant a fourth point if the CoT is clearly and logically structured, with each step building on the last. The reasoning should be easy to follow, focused on solving the user's question, and reasonably efficient, though not necessarily optimal in brevity or precision. It's acceptable to stop the reasoning early if the remaining steps are obvious or strongly implied.
- Bestow a fifth point if the CoT provides a complete, correct, and concise line of reasoning that leads efficiently to the final answer. It should include all necessary intermediate steps and be of appropriate length—neither too brief to omit key logic nor too long to include redundant or tangential content—reflecting expert-level clarity, precision, and efficiency.

User: {question}

<response>{response}</response>

After examining the user's instruction and the response:

- Briefly justify your total score, up to 100 words. Focus on relevance, logical completeness, clarity, and whether only the necessary reasoning steps were included without redundancy.
- Conclude with the score using the format: "Score: <total points>"

Remember to assess from the AI Assistant perspective, utilizing web search knowledge as necessary. To evaluate the response in alignment with this additive scoring model, systematically attribute points based on the outlined criteria.

Figure 6: LLM-as-a-judge prompt format.

D Examples of Streamlined CoT

We show qualitative examples of how our method removes redundant reasoning steps while preserving essential logic and the final answer. The green part is the CoT after binary cutting, the blue part marks the restored steps in backtracking, while the red part is removed redundant part.

Question

A *normal magic square* of order n is an arrangement of the integers from 1 to n^2 in a square such that the n numbers in each row, each column, and each of the two diagonals sum to a constant, called the *magic sum* of the magic square. Compute the magic sum of a normal magic square of order 8.

Response of DeepSeek-R1

Okay, so I need to find the magic sum of a normal magic square of order 8. Hmm, let me think. A magic square is a square grid where the numbers in each row, column, and both main diagonals add up to the same number, called the magic sum.

Since it's a normal magic square, it uses the numbers from 1 to n^2 , where n is the order. In this case, n is 8, so the numbers go from 1 to 64.

First, I remember there's a formula for the magic sum of a normal magic square. Let me try to recall. I think it involves the total sum of all the numbers in the square divided by the number of rows (or columns), since each row needs to add up to the magic sum. Let me check if that makes sense.

The total sum of numbers from 1 to n^2 can be calculated using the formula for the sum of the first m integers: $\frac{m(m+1)}{2}$.

Here, m is n^2 , so the total sum would be $\frac{n^2(n^2+1)}{2}$. Then, since there are n rows, each row must add up to the total sum divided by n . So the magic sum S should be $\frac{n^2(n^2+1)}{2n}$. Simplifying that, we get $S = \frac{n(n^2+1)}{2}$.

For example, a 3x3 magic square. The magic sum should be 15. Plugging $n = 3$ into the formula: $\frac{3(3^2+1)}{2} = \frac{3(9+1)}{2} = \frac{3 \times 10}{2} = 15$. That checks out. What about a 4x4? The magic sum should be 34. Let's see: $\frac{4(4^2+1)}{2} = \frac{4(16+1)}{2} = \frac{4 \times 17}{2} = 34$.

Therefore, for $n = 8$, substituting into the formula: $S = \frac{8(8^2+1)}{2}$. First, calculate 8 squared: $8 \times 8 = 64$. Then add 1: $64 + 1 = 65$. Multiply by 8: $8 \times 65 = 520$. Then divide by 2: $\frac{520}{2} = 260$.

Wait, just to be thorough, let me recast the problem another way. Instead of using the formula, maybe I can compute the sum of numbers from 1 to 64 and then divide by 8. Let's see. The sum from 1 to 64 is $\frac{64 \times 65}{2} = \frac{4160}{2} = 2080$. Then divide by 8 (since there are 8 rows) to get the magic sum: $\frac{2080}{8} = 260$. Same result.

A sanity check: in a magic square, the magic sum is also equal to the number of the square (n) multiplied by the average of the first and last numbers. Since the numbers run from 1 to n^2 , the first number is 1 and the last is n^2 . Their average is $\frac{1+n^2}{2}$. So magic sum should be $S = n \times \frac{1+n^2}{2}$. Wait, that's the same as the formula we had earlier: $S = \frac{n(n^2+1)}{2}$. Yep, that's exactly the same. So plugging in $n = 8$: $S = 8 \times \frac{1+64}{2} = 8 \times \frac{65}{2} = \frac{8}{2} \times 65 = 4 \times 65 = 260$. Perfect, same answer. So all these different approaches lead to the same result, 260. Therefore, I can be confident that the magic sum for an 8x8 normal magic square is 260.

****Final Answer**** The magic sum of a normal magic square of order 8 is 260.

Figure 7: It can be observed that with proper CoT prefix, the SLM can infer 260 without explicit calculation

Question

Today is January 31, 2015, welcome to the 2015 "Spring Cup" Final. The calculation result of the expression $\frac{2015/1+2015/0.31}{1+0.31}$ is:

Response of DeepSeek-R1

Okay, so I need to calculate this expression: $\frac{2015/1+2015/0.31}{1+0.31}$. Hmm, let me break this down step by step.

First, let's look at the numerator: $\frac{2015}{1} + \frac{2015}{0.31}$. The denominator is $1 + 0.31$. Maybe I can simplify each part separately before combining them.

Starting with the numerator. The first term is $2015 \div 1$, which is straightforward. Dividing any number by 1 gives the number itself, so that's just 2015. The second term is $2015 \div 0.31$. Hmm, dividing by a decimal. Let me think how to compute that.

Dividing by 0.31 is the same as multiplying by $\frac{1}{0.31}$. Let me calculate $1 \div 0.31$. Well, $0.31 \times 3 = 0.93$, and 0.31×3.2 would be 0.992. So $1 \div 0.31$ is approximately 3.2258. But I should probably do this more accurately. Let's do the division: $1 \div 0.31$.

Alternatively, I can write 0.31 as a fraction, which is $\frac{31}{100}$. So $1 \div \frac{31}{100} = \frac{100}{31}$. So $\frac{2015}{0.31} = 2015 \times \frac{100}{31}$. Let me compute that. $2015 \times 100 = 201500$. Then, $\frac{201500}{31}$. Let me do that division. $31 \times 6000 = 186000$. Then $201500 - 186000 = 15500$. $31 \times 500 = 15500$. So total is $6000 + 500 = 6500$. So $\frac{2015}{0.31} = 6500$? Wait, really? Let me check that again.

$31 \times 6500 : 31 \times 6000 = 186000$, $31 \times 500 = 15500$. Adding them together gives $186000 + 15500 = 201500$. Wait, but $\frac{2015}{0.31} = 2015 \times \frac{100}{31} = \frac{201500}{31}$. Wait, $31 \times 6500 = 201500$? Wait, 31×6500 . Let me compute $31 \times 6000 = 186000$ and $31 \times 500 = 15500$. Then $186000 + 15500 = 201500$. So yes, $31 \times 6500 = 201500$. But $2015 \times 100 = 201500$. So $\frac{201500}{31}$ is indeed 6500. So that part is correct. Therefore, $\frac{2015}{0.31} = 6500$.

So the numerator is $2015 + 6500$. Let's add those together. $2000 + 6500 = 8500$, and then the remaining 15 makes it 8515. So the numerator is 8515.

Now the denominator is $1 + 0.31 = 1.31$. So the entire expression simplifies to $\frac{8515}{1.31}$.

Hmm, so $8515 \div 1.31$. Let's see if we can compute that. Maybe there's a simplification here. Let me check if $1.31 \times 6500 = 8515$. Let me compute 1.31×6500 .

First, $1 \times 6500 = 6500$. Then 0.31×6500 . Let me compute $0.3 \times 6500 = 1950$. Then $0.01 \times 6500 = 65$. So $1950 + 65 = 2015$. Then $6500 + 2015 = 8515$. So yes, $1.31 \times 6500 = 8515$.

Therefore, $\frac{8515}{1.31} = 6500$.

Wait a minute, so the entire expression simplifies to 6500? That seems too straightforward. Let me recap to check for any mistakes. Original expression: $\frac{2015/1+2015/0.31}{1+0.31}$ Numerator: $2015 + \frac{2015}{0.31} = 2015 + 6500 = 8515$ Denominator: 1.31 $\frac{8515}{1.31} = 6500$

Yes, that seems correct. The key was recognizing that both the numerator and the denominator could be related through the same factor. Alternatively, maybe factor out 2015 from the numerator? Let me try that approach to verify.

Numerator: $2015 \left(\frac{1}{1} + \frac{1}{0.31} \right) = 2015 \left(1 + \frac{100}{31} \right) = 2015 \left(\frac{31+100}{31} \right) = 2015 \left(\frac{131}{31} \right)$

Denominator: $1.31 = \frac{131}{100}$

So the entire expression is $\frac{2015 \cdot \frac{131}{31}}{\frac{131}{100}} = 2015 \cdot \frac{131}{31} \cdot \frac{100}{131} = 2015 \cdot \frac{100}{31} = \frac{201500}{31} = 6500$.

Same result. So that confirms it. Therefore, the answer is 6500. But just to make sure, let's do the division $8515 \div 1.31$ step by step. First, write $\frac{8515}{1.31}$. To eliminate the decimal, multiply numerator and denominator by 100 to get $\frac{851500}{131}$.

Now compute $\frac{851500}{131}$. Let's see how many times 131 goes into 851500.

But earlier, we saw that $131 \times 6500 = 851500$. Because $131 \times 100 = 13100$, so $131 \times 6500 = 131 \times (65 \times 100) = (131 \times 65) \times 100$. Wait, 131×65 . Let's compute that.

$131 \times 60 = 7860$ and $131 \times 5 = 655$. So $7860 + 655 = 8515$. So $131 \times 65 = 8515$. Therefore, $131 \times 6500 = 8515 \times 100 = 851500$. Therefore, $\frac{851500}{131} = 6500$. So everything checks out. Therefore, the result of the expression is 6500. Final Answer: 6500

Figure 8: It can be observed that the model has completed the inference before the original step "Let me check that again.", and the subsequent parts become redundant for the actual answer.